E.7 Chapter 8

Automated testing is extremely important on projects other than the smallest ones. Knowing that your changes didn't break your code makes your project much more stable. The unit testing tools built into Django were covered in chapter 8.

Exercise 1

In chapter 7's exercises, you added a view called `edit_musician()`, where users could create and manage their own musician profile page. Add a test for that view. Include testing for fetching the form, submitting the form, and adding a musician with a picture. A user can edit the Musician profile they created, and so can staff and superusers. Test that those types of users can edit and that a nonowner cannot. Note that you can create a superuser with the `User.objects.create_superuser()` method.

To properly test the `edit_musician()` view, you need to create an admin to validate that they can edit a musician and you will need an image for the musician's bio-pic. The following listing shows the addition of this user and image to the test's setup.

Listing E.31 Add a method to test the `edit_musician()` view

```
# RiffMates/bands/tests.py
...
class TestBands(TestCase):
    def setUp(self):
        ...

        self.PASSWORD = "notsecure"
        ...
        self.admin = User.objects.create_superuser(
            "admin", password=self.PASSWORD
        )

        # Base64 encoded version of a single pixel GIF image
        image = "R0lGODdhAQABAIABAP///wAAACwAAAAAAQABAAACAkQBADs="
        self.image = b64decode(image)
```

Add an administrator account, and keep a reference in the test object for future use.

This Base64-encoded GIF image can be used instead of loading an image from a file.

Testing `edit_musician()` is done in steps; the following listing validates the creation of a new musician.

Listing E.32 Testing the creation of a new musician

```
# RiffMates/bands/tests.py
...
class TestBands(TestCase):
    ...
    def test_edit_musician(self):
        self.client.login(username="owner",
            password=self.PASSWORD)
```

Start by logging in as a user that will own the new Musician object.